Lab 5: Static Code Analysis
Ryona Aditree Ghosh
PES1UG24AM233

1. Pylint
    a. Before

```
(venv) C:\Users\ryona\OneDrive\Documents\L5_51_parking_lot_system>   findstr
 "rated" before\pylint_report.txt
Your code has been rated at 4.10/10
```

    b. After

```
(venv) C:\Users\ryona\OneDrive\Documents\L5_51_parking_lot_system>pylint par
king_lot_system.py
************* Module parking_lot_system
parking_lot_system.py:99:0: C0304: Final newline missing (missing-final-newl
ine)

-------------------------------------------------------------------
Your code has been rated at 9.82/10 (previous run: 4.10/10, +5.73)
```

```
(venv) C:\Users\ryona\OneDrive\Documents\L5_51_parking_lot_system>pylint par
king_lot_system.py

-------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 9.82/10, +0.18)
```

2. Bandit
    a. Before

```
Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 2
                Medium: 1
                High: 2
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 5
```

b. After

```
(venv) C:\Users\ryona\OneDrive\Documents\L5_51_parking_lot_system>bandit -r
parking_lot_system.py
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.11.9
Run started:2026-10-28 03:26:09.211077+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 68
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
```

3. Flake8

a. Before

```
parking_lot_system.py:4:1: F401 'math' imported but unused
parking_lot_system.py:7:80: E501 line too long (82 > 79 characters)
parking_lot_system.py:8:80: E501 line too long (83 > 79 characters)
parking_lot_system.py:9:80: E501 line too long (82 > 79 characters)
parking_lot_system.py:23:80: E501 line too long (131 > 79 characters)
parking_lot_system.py:32:5: E722 do not use bare 'except'
parking_lot_system.py:37:5: F841 local variable 'overtime_fee' is assigned to but never used
parking_lot_system.py:38:35: E711 comparison to None should be 'if cond is None:'
parking_lot_system.py:47:5: F841 local variable 'base_rate' is assigned to but never used
parking_lot_system.py:88:7: W292 no newline at end of file
```

b. After

(Blank output)

```
(venv) C:\Users\ryona\OneDrive\Documents\L5_51_parking_lot_system>flake8 par
king_lot_system.py
```

4. ISSUES:

Issue	Type	Description	Fixed Approach
Shell command from string (Bandit B605, High)	Security	os.system("echo ..." + filename) starts a shell, so a crafted filename could inject commands.	Replaced with print().
Weak MD5 hash (Bandit B324, High)	Security	MD5 is broken for security use.	Switched to SHA-256.
eval on a string (Bandit B307, Medium)	Security	eval(surge_expr) runs arbitrary code.	Removed it. The function now takes a numeric multiplier and returns base_rate * surge_multiplier.
Mutable default argument (Pylint W0102)	Bug	entry_logs=[] is created once and shared across calls.	Default changed to None; the list is created inside the function.
Bare except: pass (Bandit B110, Pylint W0702)	Bug / Security	Silently hides every error, including a missing slot.	Catch KeyError, report it, and return False.
Insecure random PIN (Bandit B311, Low)	Security	random.randint is predictable.	Used secrets.randbelow(9000) + 1000.
Hardcoded and printed secret (not flagged by tools)	Security	Secret token in the source and printed to the console.	Read from an environment variable and print only whether it is set.

No validation of booked hours (not flagged by tools)	Bug	Zero or negative hours reduce the total collected.	Reject hours_booked <= 0.
global statement (Pylint W0603)	Code quality	Hard to test and reason about.	Replaced with a COLLECTIONS dict.
open() without with or encoding (Pylint W1514, R1732)	Code quality	File may not be closed if an error occurs.	Used with open(..., encoding="utf-8").
camelCase names, missing docstrings (Pylint C0103, C0116)	Style	Names not snake_case and no docstrings.	Renamed everything to snake_case and added docstrings.
Line too long (Flake8 E501)	Style	Lines of 82-83 characters, over the 79 limit.	Split the dictionaries across lines.
Trailing whitespace (Flake8 W291)	Style	Whitespace at the end of the last line.	Removed with autopep8.

5. Reflection: Static Code Analysis of parking_lot_system.py

1. What is static analysis, and what did it catch that running the program did not?

Static analysis examines source code without running it. When I ran `parking_lot_system.py`, it printed sensible output and looked fine. Pylint, Bandit and Flake8 still found a lot that running it never showed:

- `eval()` on a string, which would let anyone who controls that string run arbitrary code (Bandit B307).
- An `os.system` call built from a filename, which allows command injection (Bandit B605).

- MD5 used for hashing (Bandit B324) and `random` used for a PIN (Bandit B311).
- A mutable default argument, `entry_logs=[]`, shared across calls (Pylint W0102).
- A bare `except: pass` that hides every error (Bandit B110, Pylint W0702).

None of these crashed the program. They are problems that only appear under certain inputs or in the hands of an attacker, which is why a normal run never exposed them.

2. Which issues were the most serious, and how did you prioritise?

I used Bandit's severity ratings and fixed in order of risk. The two High findings came first: the shell command in `os.system` and the weak MD5 hash. Next was the Medium finding, `eval`, which I treated as just as important because it can run arbitrary code. I then fixed the Low findings and the Pylint bug-type warnings (mutable default, bare except). Formatting and naming came last, because they affect readability but not safety or correctness.

I committed in three rounds, one per tool, so that each commit shows one kind of fix: Bandit for security, Pylint for quality, Flake8 for formatting.

3. Were there any false positives, or warnings that were not worth fixing?

None of the Bandit findings were false positives. Each one pointed at real insecure code. The closest to a judgement call was the MD5 hash: if the hash were only a label, `usedforsecurity=False` would silence the warning, but I switched to SHA-256 because it is simpler to defend and has no downside here.

Some Pylint messages were low value on their own, such as the missing docstring and snake_case naming warnings. They do not change behaviour, but they were cheap to fix and make the code easier to read, and they were needed to reach a score above 9.5.

4. What did the tools miss?

A score of 10.00/10 does not mean the code is correct. These problems were not flagged by any tool:

- A hardcoded secret token (`BARRIER_GATE_SECRET_TOKEN_2026`) that the program also printed to the console.

- `hours_booked` was never validated, so zero or negative hours were accepted and could reduce `total_collections`.
- `"%d"` formatting truncated fractional hours (2.5 hours shown as 2).

I found these by reading the code and the program's output. Static analysis checks patterns, not whether the program's logic does what it should, so it complements testing and code review but does not replace them.

5. How would you use these tools in a real project?

I would run all three automatically, not just once by hand:

- Add them as a pre-commit hook, so issues are caught before code is committed.
- Run them in a CI pipeline on every pull request and fail the build on any Bandit finding or if the Pylint score drops below a set threshold.
- Agree on a shared config (`.pylintrc`, `setup.cfg` for Flake8) so the whole team follows the same rules.

Fixing issues early is much cheaper than finding them after release. The lab also showed that the score is a useful measure of progress (4.10 to 10.00), as long as it is paired with real tests and reviews.

6. Verification:

```
(venv) C:\Users\ryona\OneDrive\Documents\L5_51_parking_lot_system>git log --
oneline
b298d31 (HEAD -> main, origin/main, origin/HEAD) Add before and after report
s
4fa5c9e Fix Flake8 formatting issues
f2eb0e2 Fix Pylint issues, score above 9.5
554ad1f Fix Bandit security issues
f3fba6a Add files via upload
```